

CITREA · SECURITY REVIEW

Citrea

Stablecoin Bridge.

A security review of Stablecoin Bridge.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Citrea	Stablecoin Bridge	Guardian	August 29 – September 20, 2025	September 29, 2025

§ 00

Contents.

38 pages — overview, confidence ranking, scope, 20 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 19 findings	P. 09
	Remediation Review · 1 finding	P. 36
§ 05	Disclaimer	P. 37
§ 06	About Guardian	P. 38

§ 01

Overview.

Guardian reviewed Stablecoin Bridge across 2 review rounds. The final report includes 8 Low, 12 Informational findings.

AUDIT FIRM	Guardian	REPOSITORY	chainwayxyz/stablecoin-bridge
CLIENT	Citrea	METHODOLOGY	Smart contract · Manual · Main Review · Remediation Review
PROJECT	Stablecoin Bridge	MAIN REVIEW COMMIT	61690aa44cfc
PREPARED BY	Guardian	REMEDATION REVIEW COMMIT	f3d5beca3d80
AUDIT WINDOW	August 29 – September 20, 2025		
FINAL REPORT	September 29, 2025		

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	0	0	0	0	0
Medium	0	0	0	0	0
Low	8	0	4	0	4
Informational	12	0	6	0	6
All severities	20	0	10	0	10

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Stablecoin Bridge in `chainwayxyz/stablecoin-bridge`.

The scope includes 1,098 source lines across 35 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
src/Destination0USDC.sol	35	69
src/Destination0USDT.sol	35	71
src/Source0FTAdapter.sol	11	15
src/USDCRolesHolder.sol	25	33
script/ConfigSetup.s.sol	179	217
src/for_circle_takeover/Destination0USDCForTakeover.sol	43	79
src/for_circle_takeover/Source0FTAdapterForTakeover.sol	42	55
script/usdt/deploy/01_USDTDeploy.s.sol	33	40
script/usdt/deploy/02_USDTBridgeDeploy.s.sol	46	54
script/usdt/deploy/03_USDTSrcBridgeSetLzConfig.s.sol	37	51
script/usdt/deploy/04_USDTEstBridgeSetLzConfig.s.sol	37	50
script/usdt/deploy/05_USDTSrcBridgeSetPeer.s.sol	21	28
script/usdt/deploy/06_USDTEstBridgeSetPeer.s.sol	21	28
script/usdt/deploy/07_USDTSetBridgeAsMinter.s.sol	18	25
script/usdt/deploy/08_USDTAndBridgeAssignRoles.s.sol	33	40
script/usdc/deploy/02_USDCBridgeDeploy.s.sol	45	53
script/usdc/deploy/03_USDCSrcBridgeSetLzConfig.s.sol	37	50
script/usdc/deploy/04_USDCDestBridgeSetLzConfig.s.sol	37	50
script/usdc/deploy/05_USDCSrcBridgeSetPeer.s.sol	21	28
script/usdc/deploy/06_USDCDestBridgeSetPeer.s.sol	21	28
script/usdc/deploy/07_USDCSetBridgeAsMinter.s.sol	19	25
script/usdc/deploy/08_USDCAndBridgeAssignRoles.s.sol	34	41
script/usdc/for_circle_takeover/01_USDCSrcBridgePrepareTakeover.s.sol	25	31
script/usdc/for_circle_takeover/02_USDCDestBridgePrepareTakeover.s.sol	25	31
script/usdc/for_circle_takeover/03_USDCSrcBridgeSetBlockedMsgLib.s.sol	20	26
script/usdc/for_circle_takeover/04_USDCDestBridgeSetBlockedMsgLib.s.sol	20	26
script/usdc/for_circle_takeover/05_USDCSrcBridgePause.s.sol	20	26
script/usdc/for_circle_takeover/06_USDCDestBridgePause.s.sol	20	26
script/usdc/for_circle_takeover/07_USDCRemoveBridgeAsMinter.s.sol	18	27
script/usdc/for_circle_takeover/08_USDCSrcBridgeSetCircle.s.sol	20	26
script/usdc/for_circle_takeover/09_USDCProxyAdminTransfer.s.sol	18	25
script/usdc/for_circle_takeover/10_USDCTransferOwner.s.sol	22	29
script/usdc/for_circle_takeover/11_USDCRolesHolderSetCircle.s.sol	20	26
script/usdc/for_circle_takeover/unpause/USDCDestBridgeUnpause.s.sol	20	26
script/usdc/for_circle_takeover/unpause/USDCSrcBridgeUnpause.s.sol	20	26

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
L-01	Missing ProxyAdmin Ownership Transfer	Configuration	LOW	ACKNOWLEDGED	p. 9
L-02	Enabling Peers Before Granting Mint Authorization	Configuration	LOW	RESOLVED	p. 11
L-03	Pausing _credit During ULN Confirmation Window	Configuration	LOW	ACKNOWLEDGED	p. 13
L-04	USDC Recipient Blacklist	DoS	LOW	ACKNOWLEDGED	p. 15
L-05	Zero-Address Recipient	DoS	LOW	RESOLVED	p. 17
L-06	Reconciled Bridged Supply	Unexpected Behavior	LOW	RESOLVED	p. 19
L-07	Fee-On-Transfer USDT	Validation	LOW	ACKNOWLEDGED	p. 20
L-08	Missing Contract Bytecode Verification	Warning	LOW	RESOLVED	p. 22
I-01	USDC.e Proxy Initialization Can Be Front-Run	Frontrunning	INFORMATIONAL	ACKNOWLEDGED	p. 23
I-02	Missing Scripts	Configuration	INFORMATIONAL	ACKNOWLEDGED	p. 25
I-03	Consider Declaring USDC As Immutable	Gas Optimization	INFORMATIONAL	RESOLVED	p. 27
I-04	Unlicensed Contracts	Best Practices	INFORMATIONAL	RESOLVED	p. 28
I-05	Destination USDC Blacklist Initialized Empty	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 29
I-06	Manual In-Flight Message Check Is Error-Prone	Warning	INFORMATIONAL	RESOLVED	p. 30

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-07	Missing __Pausable_init Invocation	Best Practices	INFORMATIONAL	RESOLVED	p. 31
I-08	Missing Storage Gaps In Upgradeable Contracts	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 32
I-09	Single Step Ownable Module	Best Practices	INFORMATIONAL	RESOLVED	p. 33
I-10	Discrepancy In The TransparentUpgradeableProxy/ProxyAdmin	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 34
I-11	setCircle Allows Circle Address To Be Re Set	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 35

REMEDIATION REVIEW

I-01	InflightMsgCheckLzScan.sh Can Silently Succeed	Configuration	INFORMATIONAL	RESOLVED	p. 36
------	--	---------------	---------------	----------	-------

LOW **L-01**

ACKNOWLEDGED

Missing ProxyAdmin Ownership Transfer

CATEGORY	Configuration
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The bridge deployment scripts import `openzeppelin-contracts/contracts/proxy/transparent/TransparentUpgradeableProxy.sol`, which resolves via your remappings to OpenZeppelin v5.2.0. In OZ v5, `TransparentUpgradeableProxy` always deploys a fresh `ProxyAdmin` inside its constructor and stores that `ProxyAdmin`'s address as the proxy admin (immutable). The constructor argument you pass is not the proxy admin itself, but the owner of the newly created `ProxyAdmin`.

In your repo, both the USDC and USDT bridge proxies are created with the v5 TUP:

- `script/usdc/deploy/02_USDCBridgeDeploy.s.sol`
- `script/usdt/deploy/02_USDTBridgeDeploy.s.sol`

Each call like:

```
new TransparentUpgradeableProxy(
    address(impl),
    _<src|dest><USDC|USDT>BridgeProxyAdminOwner, // initialOwner of a new ProxyAdmin
    abi.encodeWithSignature("initialize(address)", _bridgeOwner)
);
```

creates a new `ProxyAdmin` contract per proxy whose `owner()` is the EOA from config (e.g., `.bridge.init.proxyAdminOwner`). From that point on, all upgrades of the bridge must go through `ProxyAdmin.upgrade/upgradeAndCall`, controlled by the `ProxyAdmin.owner()`.

Your takeover scripts for USDC correctly assume a `ProxyAdmin` is present and call it:

- `script/usdc/for_circle_takeover/01_USDCSrcBridgePrepareTakeover.s.sol`
- `script/usdc/for_circle_takeover/02_USDCDestBridgePrepareTakeover.s.sol`

However, nowhere in the codebase is the ownership of these bridge `ProxyAdmin` contracts transferred to the intended final controller (e.g., a multisig or Circle). The role handover script for USDC (`script/usdc/deploy/08_USDCAndBridgeAssignRoles.s.sol`) sets bridge `Ownable` owners/delegates and master-minter ownership, but does not transfer ownership of the bridge `ProxyAdmin`. The only ownership transfer you have is for the USDC token proxy

L-01 Missing ProxyAdmin Ownership Transfer

CONTINUED

(Circle's `AdminUpgradeabilityProxy`) in `script/usdc/for_circle_takeover/09_USDCProxyAdminTransfer.s.sol`.

This is a problem because the `ProxyAdmin.owner()` for the bridge proxies remains with the original EOA, an attacker who compromises that key (or a malicious insider) can front-run Circle's `burnLockedUSDC` call. For example:

1. Right before Circle calls `burnLockedUSDC()` on the source adapter, the attacker, in this case the `ProxyAdmin`'s owner upgrades the `SourceAdapter` (`Source0FTAdapterForTakeover`), drains all the USDC it holds, and then upgrades it again to its previous implementation.
2. Circle's `burnLockedUSDC` is executed. No USDC is burnt, but they think they have burnt the full amount.
3. At this point, all the USDC tokens were stolen by the `ProxyAdmin`'s owner, but Circle thinks they were burnt.

— RECOMMENDATION

As part of the handover process to Circle, consider transferring the ownerships of the source and destination Adapter's `ProxyAdmin` to an address controlled by Circle.

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

RESOLVED

Enabling Peers Before Granting Mint Authorization

CATEGORY Configuration

LOCATION 06_USDCDestBridgeSetPeer.s.sol; 07_USDCSetBridgeAsMinter.s.sol; 06_USDTDestBridgeSetPeer.s.sol; 07_USDTSetsBridgeAsMinter.s.sol

REVIEW Main Review

DESCRIPTION

In the current deployment flow for both USDC and USDT, the destination peer is set in `06_*DestBridgeSetPeer.s.sol` before the destination bridge is authorized to mint (USDC) or recognized as the `oftContract` (USDT), which happens in `07_*SetBridgeAsMinter.s.sol`. As soon as both peers are configured, users can send messages from the source. If the final peer is set while the destination bridge still lacks authorization, any arriving message will revert during `_credit()`.

For USDC (`src/Destination0USDC.sol`), `_credit` calls `FiatTokenV2_2.mint`, which reverts until the destination bridge proxy is a minter:

```
token_.mint(_to, _amountLD); // reverts until bridge is an authorized minter
```

For USDT (`src/Destination0USDT.sol`), `_credit` calls `crosschainMint`, which is guarded by `onlyAuthorizedSender` in `OFTExtension`. Until `oftContract` is set to the destination bridge, calls revert:

```
token_.crosschainMint(_to, _amountLD); // reverts until oftContract == dest bridge
```

The impact is that users send succeed on source (tokens are locked/charged) but deliveries fail on destination until ops finish step 07 and retry.

RECOMMENDATION

Reorder or merge steps so the destination is fully authorized before enabling traffic. The safest pattern is: run `07_*SetBridgeAsMinter.s.sol` first, then `06_*DestBridgeSetPeer.s.sol` and set the source peer last (`05_*SrcBridgeSetPeer.s.sol`) to act as the "master switch". If you keep scripts separate, add a precondition guard to `06_*DestBridgeSetPeer.s.sol`, e.g., for USDC:

```
require(FiatTokenV2_2(destUSDC).isMinter(destUSDCBridgeProxy), "dest bridge not minter");
```

L-02 Enabling Peers Before Granting Mint AuthorizationCONTINUED

and for USDT:

```
require(TetherToken0FTExtension(destUSDT).oftContract() == destUSDTBridgeProxy, "oftContract  
not set");
```

Alternatively, you can also merge "authorize destination" and "set destination peer" into a single script.

— RESOLUTION

Status: Resolved.

<https://github.com/chainwayxyz/stablecoin-bridge/commit/e5d9020648f40fc1e8d0a168f0850e2d13c00904>

LOW **L-03**

ACKNOWLEDGED

Pausing `_credit` During ULN Confirmation Window

CATEGORY Configuration

LOCATION `Destination0USDCForTakeover.sol`; `Source0FTAdapterForTakeover.sol`

REVIEW Main Review

— DESCRIPTION

The LayerZero ULN path uses a `confirmations` parameter (20 blocks set in the testnet config) so DVNs only pass a message to the destination after sufficient finality to mitigate reorgs. In the current takeover implementation, both `Destination0USDCForTakeover` and `Source0FTAdapterForTakeover` apply `whenNotPaused` to both `_debit` and `_credit`. This creates a following risk:

1. A user initiates a cross-chain transfer.
2. DVNs wait out the confirmation window on the source chain.
3. During that window, the owner pauses the bridge.
4. When the packet is finally delivered, the destination's `_credit` (called by `lzReceive`) hits `whenNotPaused` and reverts. LayerZero marks the message failed until an explicit retry after unpaused.
5. Meanwhile, the user's funds are already locked (or burned in burn-and-mint design). In the takeover flow, this amount is later burned as part of supply finalization, so the user's transfer is effectively stuck until operational recovery. It may be permanently lost to the user if the burn is completed or the `MasterMinter` role is revoked for the bridge, without minting the corresponding credit.

This behaviour contradicts the migration guidance to "pause bridging activity and reconcile in-flight bridging activity to finalize the total supply of bridged USDC on the destination chain." Pausing may prevent reconciliation by causing in-flight packets to fail right before the settlement.

— RECOMMENDATION

Remove `whenNotPaused` from `_credit` in both takeover contracts. Guard only `_debit` so already verified packets continue to settle while new sends are blocked, what "pause and reconcile in-flight" requires.

L-03 Pausing _credit During ULN Confirmation WindowCONTINUED

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

USDC Recipient Blacklist

CATEGORY	DoS
LOCATION	Destination0USDC.sol; Source0FTAdapter.sol
REVIEW	Main Review

DESCRIPTION

USDC token enforces blacklist checks on both the caller and the recipient in mint and transfer functions:

```
function mint(address _to, uint256 _amount)
external
whenNotPaused
onlyMinters
notBlacklisted(msg.sender)
notBlacklisted(_to)
returns (bool)
```

```
function transfer(address to, uint256 value)
external
override
whenNotPaused
notBlacklisted(msg.sender)
notBlacklisted(to)
returns (bool)
```

In the bridge, the source `_debit` path first consumes the user's value (locks via `transferFrom` in an Adapter or burns in a Core). The packet reaches the destination `_credit`, which mints the intended amount of tokens or transfer tokens locked beforehand, to deliver funds. If `_to` is blacklisted on the destination chain at execution time, mint reverts, causing the LayerZero receive to fail. The message remains failed pending manual retry after unblacklist, while the user's funds are already locked/burned on the source.

This failure mode occurs on any path where `_credit` mints USDC (e.g. if the source chain uses a Core minter or after takeover when native USDC is the destination token). Thus, any `_credit` that mints/transfer USDC is susceptible to recipient-blacklist reverts. This is particularly risky during the takeover - if supply is finalising (locked source balance is burned) before resolving failed credits, affected users can end up without funds on the destination, even though total supply is conserved. This also contradicts the intended

L-04 USDC Recipient Blacklist

CONTINUED

"lossless delivery" property for cross-chain messages: the application reverts at the destination and forces manual intervention, despite the source having already consumed value.

— RECOMMENDATION

Make `_credit` non-reverting. Wrap mint and transfer logic implemented by `_credit` functions in try/catch blocks. On failure, queue a pending credit to be claimable after unblacklist. For bridge wrapped by the adapter, override the `_credit` function to introduce the mentioned logic.

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

RESOLVED

Zero-Address Recipient

CATEGORY	DoS
LOCATION	Destination0USDC.sol; Destination0USDCForTakeover.sol
REVIEW	Main Review

DESCRIPTION

Both `Destination0USDC` and `Destination0USDCForTakeover` call `token.mint(_to, _amountLD)` in `_credit` without remapping a zero recipient. In USDC implementations, minting tokens to `address(0)` reverts. The bridge also performs no preflight check to reject `to == 0x0`. As a result:

1. The source `_debit` first consumes value (locks via `transferFrom` in Adapter or burns in Core).
2. The LZ packet arrives. Destination `_credit` tries `mint(0x0, amount)` and reverts.
3. The LayerZero message is marked failed until manual retry with a corrected recipient, while the source value remains locked/burned.

This is inconsistent with LayerZero's OFT defaults and symmetrical implementation for USDT, which remap `0x0` to `0xdead` inside `_credit` to preserve "lossless delivery" and avoid receive-time reverts.

```
function _credit(
    address _to,
    uint256 _amountLD,
    uint32 /*_srcEid*/
) internal virtual override returns (uint256 amountReceivedLD) {
    if (_to == address(0x0)) _to = address(0xdead); // _mint(...) does not support address(0x0)
    // @dev Default OFT mints on dst.
    _mint(_to, _amountLD);
    // @dev In the case of NON-default OFT, the _amountLD MIGHT not be == amountReceivedLD.
    return _amountLD;
}
```

RECOMMENDATION

In `_credit`, remap zero-address recipient to a burn sink (`0xdead`) before minting. Consider disallowing address-zero transfers preflight.

L-05 Zero-Address RecipientCONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/chainwayxyz/stablecoin-bridge/commit/cf3edbf045de467cc04ac15b73dbe686f174f140>

LOW **L-06**

RESOLVED

Reconciled Bridged Supply

CATEGORY	Unexpected Behavior
LOCATION	Source0FTAdapterForTakeover.sol
REVIEW	Main Review

DESCRIPTION

Circle's standard states: "Circle and the third-party team will jointly coordinate to burn an amount of native USDC locked in the bridge contract on the origin chain that equals the supply of bridged USDC."

In the current implementation of `Source0FTAdapterForTakeover`, the `burnLockedUSDC` burns the entire USDC balance held by the adapter, not the reconciled amount that corresponds to the bridged token supply at the takeover.

```
function burnLockedUSDC() external onlyCircle {  
    uint256 balance = innerToken.balanceOf(address(this));  
    FiatTokenV2_2(address(innerToken)).burn(balance);  
    emit BurnedLockedUSDC(msg.sender, balance);  
}
```

Because it uses the raw balance as the burn amount, any extra USDC held by the adapter (e.g. due to incorrect direct transfers) will be burned along with the actual locked collateral that backs the bridged supply. This violates the standard's requirement to burn exactly the amount equal to the outstanding bridged supply, as more native USDC may be burned than the supply represented on the destination.

RECOMMENDATION

Implement explicit internal accounting and burn only the reconciled amount. Increase a counter on each `_debit` call by the actual received delta and decrease it on every reverse unlock/credit by the amount transferred out.

RESOLUTION

Status: Resolved.

<https://github.com/chainwayxyz/stablecoin-bridge/commit/4c6c76b73fc73f578a9e824fe6688fe94cdc8225>

LOW **L-07**

ACKNOWLEDGED

Fee-On-Transfer USDT

CATEGORY	Validation
LOCATION	SourceOFTAdapterForTakeover.sol; SourceOFTAdapter.sol
REVIEW	Main Review

DESCRIPTION

On the source chain, the OFT Adapter's `_debit` locks USDT by pulling it from the user.

```
function _debit(
    address _from,
    uint256 _amountLD,
    uint256 _minAmountLD,
    uint32 _dstEid
) internal virtual override returns (uint256 amountSentLD, uint256 amountReceivedLD) {
    (amountSentLD, amountReceivedLD) = _debitView(_amountLD, _minAmountLD, _dstEid);
    // @dev Lock tokens by moving them into this contract from the caller.
    innerToken.safeTransferFrom(_from, address(this), amountSentLD);
}
```

The default implementation assumes lossless ERC-20 and sets `amountReceivedLD` equal to `amountSentLD` from `_debitView`, without verifying what was actually received.

```
function _debitView(
    uint256 _amountLD,
    uint256 _minAmountLD,
    uint32 /*_dstEid*/
) internal view virtual returns (uint256 amountSentLD, uint256 amountReceivedLD) {
    // @dev Remove the dust so nothing is lost on the conversion between chains with different
    decimals for the token.
    amountSentLD = _removeDust(_amountLD);
    // @dev The amount to send is the same as amount received in the default implementation.
    amountReceivedLD = amountSentLD;
    // @dev Check for slippage.
    if (amountReceivedLD < _minAmountLD) {
        revert SlippageExceeded(amountReceivedLD, _minAmountLD);
    }
}
```

L-07 Fee-On-Transfer USDT

CONTINUED

If the underlying USDT token has fee-on-transfer enabled, the adapter on the source chain receives only the requested amount minus the token's transfer fee, but the message still carries an amount equal to the originally requested value. The destination `_credit` then mints the full requested amount, meaning the tokens minted on the destination exceed the collateral actually locked on the source by the size of the fee. Repeated transfers accumulate a solvency gap on the adapter. Over time, this gap can leave the adapter under-collateralized, causing return path redemptions to fail.

— RECOMMENDATION

In the adapter's `_debit` function, calculate the balance delta and use it as the amount to encode into the message. This ensures the destination mints exactly what the adapter actually received.

— RESOLUTION

Status: Acknowledged.

<https://github.com/chainwayxyz/stablecoin-bridge/commit/e8c200148f09f4732a5c7a94acd27f1c753c4411>

LOW **L-08**

RESOLVED

Missing Contract Bytecode Verification

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The deployment flow broadcasts implementations and proxies and then persists the resulting addresses to config, but it never verifies the deployed bytecode on explorers nor asserts that the on-chain code matches the expected build. For example, `script/usdc/deploy/02_USDCBridgeDeploy.s.sol` deploys `Source0FTAdapter` and a `TransparentUpgradeableProxy` and then writes the proxy address into the TOML, but performs no explorer/source verification (`--verify`, `forge verify-contract`).

The USDT script similarly deploys an implementation via raw bytecode and `assembly { create(...) }` without any follow-up attestation.

RECOMMENDATION

Implement explorer verification in the runbook for each network (implementation + proxy + admin path): For example, run with `--verify` (and the per-chain API key) immediately after/bundled with `broadcast`. For the scripts that use multichain deployments like `script/usdc/deploy/02_USDCBridgeDeploy.s.sol` consider following Foundry's multichain deployment guide: <https://getfoundry.sh/forge/deploying/#multi-chain-deployments>

RESOLUTION

Status: Resolved.

<https://github.com/chainwayxyz/stablecoin-bridge/commit/61380ca64fcc5fc8f56e4a0c6d1509010e87e41b>

INFORMATIONAL I-01

ACKNOWLEDGED

USDC.e Proxy Initialization Can Be Front-Run

CATEGORY	Frontrunning
LOCATION	01_USDCDeploy.sh; deploy-fiat-token.s.sol
REVIEW	Main Review

DESCRIPTION

The USDC deployment script `deploy-fiat-token.s.sol`, invoked by `01_USDCDeploy.sh` from Circle's stablecoin-evm repository at commit `c8c31b2`, performs the proxy deployment and subsequent multi-version initializations as separate transactions under `vm.startBroadcast(deployerPrivateKey)`, enabling an attacker to frontrun the unpermissioned `initialize` calls on the newly deployed `FiatTokenProxy` after its transaction executes but before the deployer's init transactions are run, allowing the attacker to hijack ownership by calling `initialize` first with malicious parameters.

Specifically, the script deploys or reuses the implementation via `FiatTokenV2_2 fiatTokenV2_2 = getOrDeployImpl(_impl)` which safely initializes the implementation using a temporary upgrader to disable its own reinitialization, then deploys the proxy with `FiatTokenProxy proxy = new FiatTokenProxy(address(fiatTokenV2_2))` in one transaction, followed by separate transactions for `proxy.changeAdmin(proxyAdmin)` and the casts to `FiatTokenV2_2 proxyAsV2_2 = FiatTokenV2_2(address(proxy))` to invoke `proxyAsV2_2.initialize()` which delegates to the implementation's V1 `initializer` setting the owner and other roles and then individual calls to `proxyAsV2_2.initializeV2()`, `proxyAsV2_2.initializeV2_1()` and `proxyAsV2_2.initializeV2_2()`.

Since Foundry broadcasts each of these operations as distinct transactions and the initializers are not permissioned as per Circle's design to allow one-time setup, an attacker monitoring the public mempool can detect the proxy deployment transaction, predict its address using the deployer's nonce and the `CREATE` opcode determinism and submit a competing transaction with higher gas fees to execute first, casting the proxy to `FiatTokenV2_2` and calling `initialize` with the attacker's address as owner to set `initialized = true` in the proxy's storage and claim all roles before the deployer's transactions process.

If successful, the deployer's init calls should revert with "already initialized" errors and would require a new deployment/re-run of the script.

I-01 USDC.e Proxy Initialization Can Be Front-Run

CONTINUED

— RECOMMENDATION

Be aware of this risk. Consider also refactoring the `deploy-fiat-token.s.sol` script to deploy a temporary upgrader contract that atomically deploys the proxy, changes the admin, and calls all initializers in a single transaction, eliminating the multi-tx window.

— RESOLUTION

Status: Acknowledged.

<https://github.com/chainwayxyz/stablecoin-bridge/commit/67838e2ce131165308b96927caac16a4608b6e9b>

INFORMATIONAL I-02

ACKNOWLEDGED

Missing Scripts

CATEGORY	Configuration
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The repository implements the takeover mechanics but does not provide scripts to execute the critical on-chain calls at upgrade time, as required by the project's own `auditors_guide.md` (which states that every on-chain action in the Circle standard must have a corresponding script).

1. No script calls `burnLockedUSDC()` on the source bridge: The takeover implementation `src/for_circle_takeover/SourceOFTAdapterForTakeover.sol` exposes `burnLockedUSDC`. This is the exact entrypoint Circle requires on the source chain to burn the locked USDC that backs the finalized bridged supply. However, under `script/usdc/for_circle_takeover/` there is no script that invokes this function from the configured `circle` address. Existing takeover scripts cover preparing new impls, pausing/unpausing, removing minter, setting `circle` and transferring `ProxyAdmin`, but they never perform the actual burn.
2. No script triggers `USDCRolesHolder.transferUSDCRoles(address)` (Circle-side finalization). `src/USDCRolesHolder.sol` correctly exposes `transferUSDCRoles`. You deploy the roles holder and set Circle with:

- `script/usdc/for_circle_takeover/10_USDCTransferOwner.s.sol`
- `script/usdc/for_circle_takeover/11_USDCRolesHolderSetCircle.s.sol`

But there is no script that calls `transferUSDCRoles(<Circle owner>)` to actually hand over the Implementation Owner to Circle.

RECOMMENDATION

Add two minimal, purpose-built takeover scripts and wire them into the documented runbook:

1. `** 12_USDCSrcBridgeBurnLockedUSDC.s.sol **` (run on source chain, from the configured `circle` address):
 - Fork `srcRPC`.
 - Call `SourceOFTAdapter(srcUSDCBridgeProxy).burnLockedUSDC()`.
 - Recommended pre-checks: ensure the bridge is paused, log `innerToken.balanceOf(proxy)`.
2. `** 13_USDCRolesHolderTransferUSDCRoles.s.sol **` (run on destination chain, from the

I-02 Missing Scripts

CONTINUED

configured `circle` address):

- Fork `destRPC`.
- Obtain roles holder from `FiatTokenV2_2(destUSDC).owner()`.
- Call `USDCRolesHolder(rolesHolder).transferUSDCRoles(<CIRCLE_IMPLEMENTATION_OWNER>)`.
- Ensure your separate `ProxyAdmin` transfer (`09_USDCProxyAdminTransfer.sol`) has already been executed.

Also update the guide to include these steps in the exact sequence (pause → remove minter → set Circle → burnLockedUSDC → transfer ProxyAdmin → transferUSDCRoles → unpause if applicable).

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

RESOLVED

Consider Declaring USDC As Immutable

CATEGORY	Gas Optimization
LOCATION	USDCRolesHolder.sol
REVIEW	Main Review

DESCRIPTION

In the `USDCRolesHolder` contract, the `usdc` state variable is assigned in the constructor to reference the `FiatTokenV2_2` interface at the provided `usdcProxy` address and is never modified thereafter, as there are no setter functions or internal assignments that alter its value post-deployment. This variable is used solely for read operations, such as in the `transferUSDCRoles` function where it calls `usdc.transferOwnership(_owner)`, making it a prime candidate for immutability. Declaring it as a regular public variable incurs unnecessary storage reads during runtime, which consume gas, whereas an immutable variable is embedded directly into the bytecode at deployment time, eliminating storage access costs.

RECOMMENDATION

Modify the `usdc` declaration to `FiatTokenV2_2 public immutable usdc;` and assign it directly in the constructor without changes to the existing logic, ensuring the value is set at deployment and remains constant thereafter.

RESOLUTION

Status: Resolved.

<https://github.com/chainwayxyz/stablecoin-bridge/commit/5ea30e55c8ada71ca2490061cdd0f6ec26badbcb>

INFORMATIONAL **I-04**

RESOLVED

Unlicensed Contracts

CATEGORY	Best Practices
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

Several in-scope contracts are marked with

```
// SPDX-License-Identifier: UNLICENSED
```

which grants no rights to use, modify, or distribute the code. This ambiguity can deter integrators and developers from using or contributing to the project.

Additionally, mixed or missing license headers across contract (e.g. some files unlicensed, others permissively licensed) create inconsistency, making it unclear which terms apply to which components and how the code may be combined or redistributed.

RECOMMENDATION

Choose and apply a clear open-source license. Update every Solidity file's SPDX header to the chosen identifier (keeping third-party code under its original license). Common options:

- MIT – permissive, minimal restrictions.
- Apache-2.0 – permissive, explicit patent grant.
- GPL-3.0 – copyleft, derivatives must remain open-source.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-05

ACKNOWLEDGED

Destination USDC Blacklist Initialized Empty

CATEGORY	Warning
LOCATION	FiatToken.sol
REVIEW	Main Review

DESCRIPTION

The USDC deployment script (`script/usdc/deploy/01_USDCDeploy.sh`) force-sets an empty blacklist before running Circle's deployer by writing `[]` into `blacklist.remote.json`: `echo "[]" > blacklist.remote.json`

Circle's FiatToken contracts actively maintain a blacklist on production chains (e.g., Arbitrum shows a designated `Blacklister` at `[0x13F2A44FaD26c2cc25d3e3b869364142ce5995Bb]` (<https://arbiscan.io/address/0x13F2A44FaD26c2cc25d3e3b869364142ce5995Bb>) and USDC runs at `[0xaf88d065e77c8cC2239327C5EDb3A432268e5831]` (<https://arbiscan.io/token/0xaf88d065e77c8cC2239327C5EDb3A432268e5831>)). Initializing the destination chain with an empty blacklist means addresses currently blocked on other chains will not be blocked on the new destination at launch. During this divergence window, a recipient blacklisted on Arbitrum (or Ethereum) can receive mints and transfer on the destination chain until the list is manually synced, which could be a policy/compliance gap.

RECOMMENDATION

Consider checking with the Circle's team how to proceed with the blacklist. Blacklisting all the addresses in the destination chain would imply very high gas costs.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-06

RESOLVED

Manual In-Flight Message Check Is Error-Prone

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The takeover runbook instructs operators to “wait and check LayerZero Scan” to ensure no in-flight messages exist before pausing the bridges. This off-chain, manual verification is easy to skip or perform inconsistently. If operators pause while messages are still in flight, subsequent deliveries will hit `whenNotPaused` guards in the takeover implementations (e.g., `Destination0USDCForTakeover._credit`) and revert, creating avoidable failed executions that require retries/unpausing.

The LayerZero Scan API already exposes a per-OApp feed, GET `/messages/oapp/{eid}/{address}` with optional limit, start, end and nextToken, that returns statuses such as `INFLIGHT`, `CONFIRMING`, `PAYLOAD_STORED`, `BLOCKED`, `FAILED`, and `DELIVERED`. Relying solely on human diligence instead of this endpoint risks pausing with unresolved messages.

RECOMMENDATION

Before executing pause scripts, add an automated pre-check that queries GET `/messages/oapp/{eid}/{address}` for both destination and source bridge proxies and aborts if any `non-DELIVERED` messages are returned.

RESOLUTION

Status: Resolved.

<https://github.com/chainwayxyz/stablecoin-bridge/commit/4822498d2d86004f81bf81f321d3f066b5463070>

INFORMATIONAL **I-07**

RESOLVED

Missing `__Pausable_init` Invocation

CATEGORY	Best Practices
LOCATION	SourceOFTAdapterForTakeover.sol
REVIEW	Main Review

DESCRIPTION

`SourceOFTAdapterForTakeover` inherits `PausableUpgradeable` but its `initialize` function does not call `__Pausable_init`.

```
function initialize(address _delegate) public initializer {  
    __OFTCore_init(_delegate);  
    __Ownable_init(_delegate);  
}
```

In OpenZeppelin's upgradeable contracts pattern, every inherited module's initializer should be invoked from the proxy initializer (e.g. `__Pausable_init` when inheriting `PausableUpgradeable`). Omitting it deviates from the recommended initialization procedure for upgradeable contracts.

RECOMMENDATION

Call `__Pausable_init` inside the `initialize` function.

RESOLUTION

Status: Resolved.

<https://github.com/chainwayxyz/stablecoin-bridge/commit/ff5f596275eefca65ee553eb204b80444db70786>

INFORMATIONAL I-08

ACKNOWLEDGED

Missing Storage Gaps In Upgradeable Contracts

CATEGORY	Best Practices
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Upgradeable contracts in scope do not declare a storage gap. OpenZeppelin recommends reserving unused storage slots at the end of upgradeable contracts to reduce the risk of storage layout conflicts in future versions (e.g. when adding new state variables, changing inheritance or upgrading dependencies storage). Without a gap, adding variables later may shift storage and corrupt state in existing proxies.

— RECOMMENDATION

Add a storage gap to each upgradeable contract even if it currently uses only immutable variables. When adding state later, consume slots from the gap and keep prior layout preserved.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-09

RESOLVED

Single Step Ownable Module

CATEGORY	Best Practices
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The bridge and adapter contracts rely on an owner account to perform sensitive lifecycle actions (configure peers, pause/unpause, set Circle takeover roles, assign/revoke mint/burn permissions). This design allows operators to react quickly and coordinate the Bridged USDC takeover. However, the contracts use single-step `Ownable` module, where ownership transfer finalize immediately and privileged functions execute without delay.

This creates a potential risk, where a compromised key, mistaken transfer, or rushed action could instantly allow to:

- set peer to a malicious OApp (granting mint authority on the destination bridge),
- change takeover addresses (`setCircle`) or call `burnLockedUSDC` at the wrong time,
- push unreviewed LayerZero security/library/DVN config changes,
- pause/unpause contracts in an undesirable time.

Because ownership changes take effect immediately, there is no review window for monitoring to catch bad actions and no explicit acceptance step by the new owner.

RECOMMENDATION

- Consider implementing `Ownable2Step` module (if feasible and compatible with the Bridged USDC Standard), so that ownership transfers require explicit acceptance by the new owner.
- Process non-emergency privileged calls (`setPeer` , DVN/library config, `setCircle` , minter role changes) through the timelock.
- Set the owner to a multisig behind a timelock.
- Override the `renounceOwnership` function to prevent accidental loss of ownership.
- Implement a timelock on the `setPeer` function and monitor pending changes.

RESOLUTION

Status: Resolved.

<https://github.com/chainwayxyz/stablecoin-bridge/commit/309d8cf052da4b529c434ec5a0ada6641b908d56>

INFORMATIONAL **I-10**

ACKNOWLEDGED

Discrepancy In The TransparentUpgradeableProxy/ProxyAdmin

CATEGORY Warning
LOCATION 09_USDCProxyAdminTransfer.s.sol
REVIEW Main Review

DESCRIPTION

The `TransparentUpgradeableProxy` contracts deployed in all the scripts except `script/usdc/deploy/01_USDCDeploy.sh` are a v5+ Openzeppelin version: `import "openzeppelin-contracts/contracts/proxy/transparent/TransparentUpgradeableProxy.sol";`. `TransparentUpgradeableProxy` v5+ use an immutable admin and auto-deploys a `ProxyAdmin` in its constructor.

However, USDC uses Circle's `FiatTokenProxy` / Openzeppelin's v4 `AdminUpgradeabilityProxy`: `[stablecoin-evm/contracts/upgradeability/AdminUpgradeabilityProxy.sol at master · circlefin/stablecoin-evm](https://github.com/circlefin/stablecoin-evm/blob/master/contracts/upgradeability/AdminUpgradeabilityProxy.sol)` that uses a mutable admin slot with constructor allowing direct admin swaps via `changeAdmin(newAdmin)` only callable by the current admin.

RECOMMENDATION

Merely informative. Be aware that the USDC token address in the source chain is the only contract that is not behind a TUP.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-11**

ACKNOWLEDGED

setCircle Allows Circle Address To Be Re Set

CATEGORY	Warning
LOCATION	Source0FTAdapterForTakeover.sol
REVIEW	Main Review

— DESCRIPTION

The `setCircle` function allows the owner to set the address for `circle`. However the function does not prohibit the ability to call `setCircle` again once the contract is given the zero allowance `Minter` role.

According to the standard,

> Be only callable by an address that Circle specifies closer to the time of the upgrade. Note that this address will not necessarily be the same address that is specified to call `transferUSDCRoles`.

The above describes restrictions according to the `burnLockedUSDC` function. This function is callable by the address that is set as `circle` in the `setCircle` function. Therefore the code does not fully comply with the standard since the owner is allowed to change the `circle` address after it was already set.

— RECOMMENDATION

Consider changing the logic of `setCircle` to

```
function setCircle(address _circle) external onlyOwner {
    require(circle == address(0), "circle already set");
    require(_circle != address(0), "zero address");
    circle = _circle;
    emit CircleSet(_circle);
}
```

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-01

RESOLVED

InflightMsgCheckLzScan.sh Can Silently Succeed

CATEGORY	Configuration
LOCATION	InflightMsgCheckLzScan.sh
REVIEW	Remediation Review

DESCRIPTION

The takeover helper `script/usdc/for_circle_takeover/InflightMsgCheckLzScan.sh` relies on `curl` and `jq` to ensure there are no LayerZero messages in flight before pausing the bridge. When the HTTP call fails or returns malformed JSON, `jq` exits with a non-zero status and nothing is written to stdout, yet the following if:

```
if echo "$response" | jq -e '.data[]? | select(.status.name == "INFLIGHT" or .status.name == "CONFIRMING")' > /dev/null; then
```

treats that failure exactly like “no inflight messages” and the function returns 0, so the script prints “SUCCESS: All checks passed.” This creates a false sense of safety as subsequent pause scripts will continue even though the bridge’s state is unknown, making it possible that real inflight transfers get frozen mid-upgrade.

RECOMMENDATION

Fail fast on tooling errors. Add `set -euo pipefail` to the shell prologue, capture the exit status of both `curl` and `jq` and abort unless both succeed. If either command fails, emit a clear error and return a non-zero exit code so the pausing scripts refuse to continue.

RESOLUTION

Status: Resolved.

<https://github.com/chainwayxyz/stablecoin-bridge/commit/7df209891507f295c24f3f6955230000a02b52dd>

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.